1. (3 Punkte) a. Benenne im folgenden Code alle Beispiele für die folgenden Begriffe: globale Variable, lokale Variable, Parameter und Argument.
   b. Was wird ausgegeben?

```
k = 10
def doit(y):
    r = 5
    w = k + y + r
    return w

z = doit(3) + k
print(z)
```

Lösung:

a. k, z sind globale Variablen, y, r, w sind lokale Variablen, 3 ist Argument, y ist Parameter.
b. 28

2. (1 Punkt) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben.

```
'''
returns: string, die Zeichenfolge 'x.'

>>> func2()
'x.'
'''
```

Lösung:

```
def func2():
    return 'x.'
```

3. (1 Punkt) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben.

```
'''
returns: None,  Gibt die Zeichenfolge 'x.' aus

>>> func1()
x.
'''
```

Lösung:

```
def func1():
    print('x.')
```

4. (3 Punkte) Gib an, was ausgegeben wird. Tritt ein Fehler auf, gib an, in welcher Zeile und warum.

```
def f(x):
    print(x + 1)

def g(x):
    return x + 1

a = f(3)              # Zeile 7
b = g(3)              # Zeile 8
print(a, b)           # Zeile 9
print(g(g(1)))        # Zeile 10
f(g(1))               # Zeile 11
print(f(1) + 1)       # Zeile 12
```

Lösung:

```
4           Zeile 7: f gibt 4 aus, a bekommt None
None 4      Zeile 9
3           Zeile 10: g(1) = 2, g(2) = 3
3           Zeile 11: g(1) = 2, f gibt 3 aus
2           Zeile 12: f(1) gibt 2 aus und liefert None,
            danach Fehler (TypeError): None + 1 ist nicht möglich
```

5. (2 Punkte) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben.

```
'''
x: nicht leerer string
returns: String, der nur aus dem ersten und letzten Zeichen von x besteht.

>>> teil('Hallo')
'Ho'

>>> teil('H')
'HH'
'''
```

Lösung:

```
def teil(x):
    return x[0]+x[-1]
```

6. (2 Punkte) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben.

```
'''
s string, k >= 0, k optional mit default 1
returns: string, die ersten k zeichen von s

>>> make('Hallo')
'H'
>>> make('Hallo',3)
'Hal'
'''
```

Lösung:

```
def make(s, k=1):
    return s[:k]
```

7. (3 Punkte) Gegeben ist die folgende Funktion:

```
def preis(netto, mwst=19, rabatt=0):
    return netto * (100 + mwst) // 100 - rabatt
```

Gib an, was ausgegeben wird, bzw. ob ein Fehler auftritt.

```
print(preis(100))
print(preis(100, 7))
print(preis(100, rabatt=10))
print(preis(rabatt=5, netto=200))
print(preis(50, 7, 3))
print(preis(mwst=7))
```

Lösung:

```
119     mwst und rabatt haben ihre Standardwerte
107     mwst = 7
109     mwst = 19, rabatt = 10
233     Schlüsselwortargumente, Reihenfolge egal: 238 - 5
50      53 - 3
Fehler (TypeError): für netto gibt es keinen Standardwert
```

8. (1 Punkt) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben.

```
'''
x: int
returns 'A' falls x < 20
        'B' falls 20 <= x < 30
        'C' falls 30 <= x

>>> func(15)
'A'
>>> func(25)
'B'
>>> func(35)
'C'
'''
```

Lösung:

```
def func(x):
    if x < 20: return 'A'
    if 20 <= x < 30: return 'B'
    return 'C'
```

9. (3 Punkte) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben. Die Funktion soll mit `return` verlassen werden, sobald das Ergebnis feststeht.

```
'''
a: Liste mit Zahlen
returns: True, wenn die Liste aufsteigend sortiert ist, sonst False

>>> ist_sortiert([1, 3, 3, 7])
True
>>> ist_sortiert([1, 5, 2, 7])
False
>>> ist_sortiert([])
True
'''
```

Lösung:

```
def ist_sortiert(a):
    for i in range(len(a) - 1):
        if a[i] > a[i + 1]:
            return False        # erstes Paar in falscher Reihenfolge
    return True
```

10. (4 Punkte) Schreibe eine Funktion `statistik(noten)`, die für eine Liste von Schulnoten (1 bis 6) die beste Note, die schlechteste Note und den Durchschnitt als Tuple zurückgibt.

    Rufe die Funktion für die Liste `[2, 3, 1, 4, 2]` auf, entpacke das Ergebnis in drei Variablen und gib es in der Form `beste: 1, schlechteste: 4, Schnitt: 2.4` aus.

Lösung:

```
def statistik(noten):
    return min(noten), max(noten), sum(noten) / len(noten)

beste, schlechteste, schnitt = statistik([2, 3, 1, 4, 2])
print(f'beste: {beste}, schlechteste: {schlechteste}, Schnitt: {schnitt}')
```

(Die beste Note ist die kleinste Zahl, deshalb `min`.)

11. (1 Punkt) Was wird ausgegeben?

```
x = 10
def doit(k):
    global x
    x = 5
    z = 7
    w = k + z
    return w

z = doit(3) + x
print(z)
```

Lösung: 15

12. (4 Punkte) Was erscheint auf der Konsole?

```
def a(x,y):
    return x + 2 * y
def b(x):
    return x - 3
def c(x,y):
    return (x + 3) // y
def d(x,y):
    return x > b(y)
z = c(a(3,1),b(7))
print(d(z,5),z)
```

Lösung: False 2

13. (4 Punkte) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben.

```
'''
n: int, n > 0
returns: die Summe der echten Teiler von n. Echte Teiler sind alle Teiler der Zahl,
         außer die Zahl selbst. Beispiel: die Summe der echten Teiler
         von 12 ist: 1+2+3+4+6 = 16

>>> echteTeilerSumme(12)
16
'''
```

Lösung:

```
def echteTeilerSumme(n):
    summe = 0
    for k in range(1,n):
        if n % k == 0:
            summe +=k
    return summe
```

14. (3 Punkte) Eine natürliche Zahl heißt *vollkommen*, wenn sie gleich der Summe ihrer echten Teiler ist, z.B. 6 = 1+2+3. Verwende die Funktion `echteTeilerSumme` aus der vorigen Aufgabe.

    (a) (1 Punkt) Schreibe eine Funktion `ist_vollkommen(n)`, die `True` zurückgibt, wenn `n` vollkommen ist, sonst `False`.

    (b) (2 Punkte) Schreibe eine Funktion `vollkommene_zahlen(bis)`, die eine Liste aller vollkommenen Zahlen von 1 bis `bis` zurückgibt. `vollkommene_zahlen(500)` liefert `[6, 28, 496]`.

Lösung:

```
def ist_vollkommen(n):
    return echteTeilerSumme(n) == n

def vollkommene_zahlen(bis):
    ergebnis = []
    for k in range(1, bis + 1):
        if ist_vollkommen(k):
            ergebnis.append(k)
    return ergebnis
```

15. (4 Punkte) Gegeben sind die Signaturen einiger Java-Methoden:

```
static int maximum(int a, int b)
static void begruessen(String name)
static boolean istGerade(int n)
static double kreisflaeche(double r)
```

(a) (2 Punkte) Gib für jede Methode an, welche Parameter (mit Datentyp) sie hat und ob und welchen Wert sie zurückgibt.

(b) (2 Punkte) Schreibe `maximum` und `begruessen` als Python-Funktionen mit Typhinweisen. `begruessen('Ida')` soll `Hallo Ida!` ausgeben.

Lösung:

Methode	Parameter	Rückgabewert
`maximum`	`a`, `b` vom Typ `int`	ein `int`
`begruessen`	`name` vom Typ `String`	keiner (`void`)
`istGerade`	`n` vom Typ `int`	ein `boolean`
`kreisflaeche`	`r` vom Typ `double`	ein `double`

```
def maximum(a: int, b: int) -> int:
    if a > b:
        return a
    return b

def begruessen(name: str):
    print(f'Hallo {name}!')
```

16. (3 Punkte) Die folgende Funktion soll die größte von drei Zahlen zurückgeben.

```
def maximum(a, b, c):
    '''
    >>> maximum(1, 2, 3)
    3
    >>> maximum(3, 2, 1)
    3
    >>> maximum(1, 3, 2)
    3
    '''
    if a > b and a > c:
        return a
    if b > a:
        return b
    return c
```

(a) (1 Punkt) Welcher Doctest schlägt fehl, und was liefert die Funktion dort?

(b) (1 Punkt) Korrigiere die Funktion.

(c) (1 Punkt) Warum hätte der Test `maximum(3, 2, 1)` allein den Fehler nicht gefunden?

Lösung:

a) `maximum(1, 2, 3)` liefert `2` statt `3`: Weil `b > a` ist, wird `b` zurückgegeben, ohne `b` mit `c` zu vergleichen.

b) Zeile `if b > a:` ersetzen durch `if b > c:`. (Mit `>=` statt `>` funktioniert es auch, wenn Zahlen gleich sind, z.B. `maximum(2, 2, 1)`.)

c) Bei `maximum(3, 2, 1)` ist `a` am größten, das erste `return` wird ausgeführt und der fehlerhafte Teil gar nicht erreicht. Gute Tests prüfen deshalb mehrere verschiedene Fälle.

17. (4 Punkte) Gegeben ist eine Liste von Tuples `(name, alter)`:

```
personen = [('Lena', 17), ('Tom', 16), ('Maximilian', 18), ('Ida', 16)]
```

(a) (2 Punkte) Schreibe eine Funktion `alter(p)`, die das Alter einer Person zurückgibt, und gib damit die Liste nach dem Alter sortiert sowie die jüngste Person aus.

(b) (1 Punkt) Gib die Person mit dem längsten Namen aus. Verwende `max` mit einer eigenen `key`-Funktion.

(c) (1 Punkt) Was gibt `print(sorted(['Kiwi', 'Apfel', 'Banane', 'Feige'], key=len))` aus?

Lösung:

```python
# a)
def alter(p):
    return p[1]

print(sorted(personen, key=alter))
print(min(personen, key=alter))        # ('Tom', 16)

# b)
def name_laenge(p):
    return len(p[0])

print(max(personen, key=name_laenge))  # ('Maximilian', 18)
```

c) `['Kiwi', 'Apfel', 'Feige', 'Banane']`. Bei gleicher Länge bleibt die ursprüngliche Reihenfolge erhalten.